Day 121 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 18, Day 3 reference

Watch: Clustering with DBSCAN, Clearly Explained!!! (StatQuest with Josh Starmer) | DBSCAN Explanation and Visualization (TheDataPost)

`day121_dbscan.py`, segment by segment, in the order `main()` uses things. Neighbourhoods, DBSCAN, the k-distance curve and the California helpers are new; Day 119's sqdist, ari, kmeans and elbow are imported unchanged. scikit-learn's DBSCAN appears only as an outside check. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: eps = the radius · min_pts = neighbours needed to be core, the point itself included · N = the list of neighbourhoods · lab = labels, NOISE = –1

Segment 0 — Setup

Imports and settings

CODE

```
HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE)
with contextlib.redirect_stdout(io.StringIO()):
    import day119_k_means as d119
sqdist, ari = d119.sqdist, d119.ari
NOISE = -1
CITIES = {"Los Angeles": (34.05, -118.24), "San Francisco": (37.77, -122.42), "Oakland": (37.80,
    -122.27),
          "San Jose": (37.34, -121.89), "San Diego": (32.72, -117.16), "Sacramento": (38.58,
              -121.49),
          "Fresno": (36.74, -119.79)}
```

- `from sklearn.datasets import fetch_california_housing` — downloads the dataset on first use and reads it from a local cache afterwards.
- `NOISE = -1` — a named constant: the code says NOISE, not a bare -1, as in scikit-learn's convention.
- `CITIES = {name: (lat, lon)}` — used only afterwards, to name the clusters found.

Segment 1 — DBSCAN

neighbours: everything within eps

MATH

$$N(x_i) = \{j : |x_i - x_j|^2 \le eps^2\}$$

CODE

```
def neighbours(X, eps, chunk=1000):
    out = []
    for s in range(0, len(X), chunk):
        D = sqdist(X[s:s + chunk], X)
        out += [np.where(row <= eps * eps)[0] for row in D]
    return out
```

- `range(0, len(X), chunk)` — start positions 0, 1000, 2000, ...: a chunk of rows at a time keeps the table at 1000 x n.

- `row <= eps * eps` — comparing squared distances with eps squared avoids a square root for every pair.
- `out += [... for row in D]` — += on a list appends every item of the new list.

dbscan: grow clusters through core points

MATH

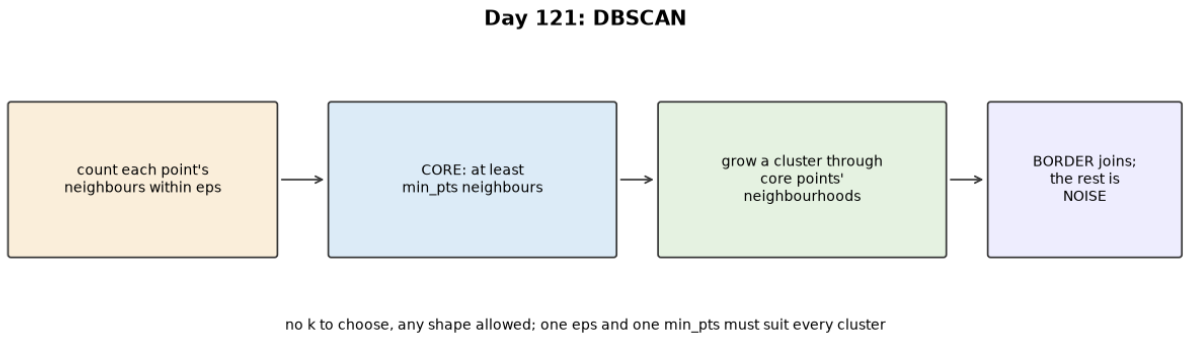
$$x \text{ core} \Leftrightarrow |N(x)| \geq min_pts; \quad y \text{ joins } x\text{'s cluster if } x \text{ is core and } y \in N(x)$$

CODE

```
def dbscan(X, eps, min_pts):
    N = neighbours(X, eps); core = np.array([len(nb) >= min_pts for nb in N])
    lab = np.full(len(X), NOISE); c = 0
    for i in range(len(X)):
        if lab[i] != NOISE or not core[i]:
            continue
        lab[i] = c; stack = [i]
        while stack:
            j = stack.pop()
            if core[j]:
                for v in N[j]:
                    if lab[v] == NOISE:
                        lab[v] = c; stack.append(v)
        c += 1
    return lab, core
```

- `np.array([len(nb) >= min_pts for nb in N])` — a True/False array: which points are core.
- `np.full(len(X), NOISE)` — every point starts as noise; growing clusters relabel them.
- `stack = [i]; ... stack.pop()` — a list used as a stack: take the last point added, look at its neighbours, add the unlabelled ones.
- `if core[j]:` — only core points pass the cluster on: border points join but stop the growth.

SYSTEM DIAGRAM



Core, border and noise.

k_distance and the knee

CODE

```
def k_distance(X, k, chunk=1000):
    out = []
    for s in range(0, len(X), chunk):
        D = np.maximum(sqdist(X[s:s + chunk], X), 0)
        out.append(np.sqrt(np.partition(D, k - 1, axis=1)[:, k - 1]))
    return np.concatenate(out)
def knee_eps(X, k):
    kd = np.sort(k_distance(X, k))[::-1]
    i = d119.elbow(list(range(len(kd))), kd)
    return float(kd[i]), kd
```

- `np.partition(D, k - 1, axis=1)[:, k - 1]` — the k-th smallest value in each row, without sorting the whole row.
- `np.sort(...)[::-1]` — sorted, then reversed: largest first.
- `d119.elbow(list(range(len(kd))), kd)` — Day 119's elbow rule, reused: the position furthest below the chord.

Segment 2 — Data

California: degrees to kilometres, and naming clusters

MATH

$$x\ (\text{km}) \approx 111.32\ \cos(\text{latitude}) \times \Delta\text{longitude}; \quad y\ (\text{km}) \approx 110.57 \times \Delta\text{latitude}$$

CODE

```
def california():
    d = fetch_california_housing(); lat, lon = d.data[:, 6], d.data[:, 7]
    km = np.c_[(lon - lon.mean()) * 111.32 * np.cos(np.radians(lat.mean())), (lat - lat.mean()) *
        110.57]
    return km, lat, lon, d.target
def cities_in(lab, c, lat, lon):
    out = []
    for name, (a, b) in CITIES.items():
        i = int(np.argmin(np.hypot((lat - a) * 110.57, (lon - b) * 111.32 *
            np.cos(np.radians(a)))))
        out += [name] if lab[i] == c else []
    return ", ".join(out) if out else "(none listed)"
def summary(lab):
    s = np.bincount(lab[lab >= 0]) if (lab >= 0).any() else np.array([], int)
    return len(s), float(np.mean(lab == NOISE)), sorted(s.tolist(), reverse=True)
```

- `(lon - lon.mean()) * 111.32 * np.cos(np.radians(lat.mean()))` — a degree of longitude shrinks with latitude: about 111 km times the cosine.
- `np.c_[a, b]` — two columns side by side.
- `out += [name] if lab[i] == c else []` — a conditional expression: add the name, or add nothing.
- `", ".join(out) if out else "(none listed)"` — an empty list is false, so the fallback text is used.

Segment 3 — The experiments

demo_check: comparing core points with scikit-learn's

CODE

```
lab, core = dbscan(X, eps, mp); sk = DBSCAN(eps=eps, min_samples=mp).fit(X)
sk_core = np.zeros(len(X), bool); sk_core[sk.core_sample_indices_] = True
```

- `sk.core_sample_indices_` — scikit-learn reports core points as a list of positions.
- `sk_core[...] = True` — turns those positions into a True/False array, comparable with ours by np.array_equal.

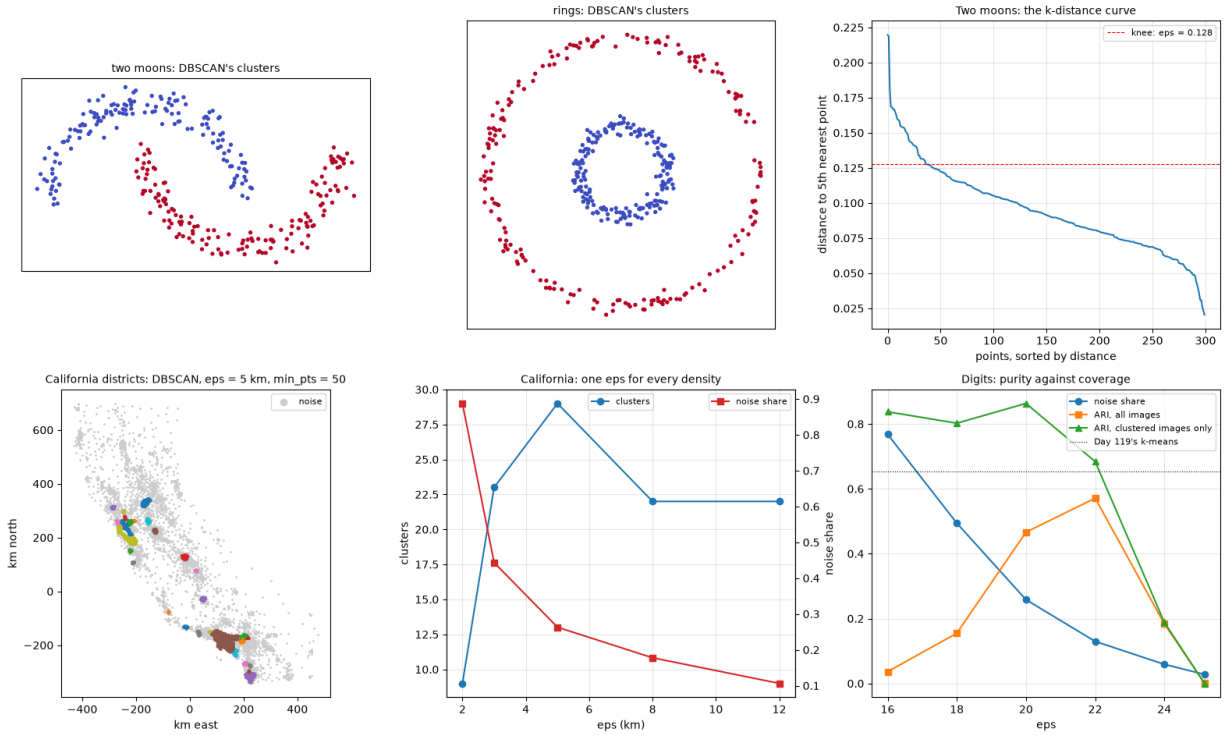
demo_density: the best of 21 settings

CODE

```
X, y = sets["wide + tight"]; best = (-1, None)
for eps in (0.1, 0.15, 0.2, 0.3, 0.5, 0.8, 1.2):
    for mp in (5, 10, 20):
        lab, _ = dbscan(X, eps, mp); a = ari(lab, y)
        if a > best[0]:
            best = (a, (eps, mp, summary(lab)))
a, (eps, mp, (k, nz, s)) = best
```

- `best = (-1, None)` — a starting score below any ARI, so the first setting always replaces it.
- `a, (eps, mp, (k, nz, s)) = best` — nested unpacking: the tuple's structure is mirrored on the left.

REAL GRAPH



Real graph from the script: moons, rings, the k-distance curve, California, the eps scan and digits.

Quick Syntax Glossary — new or reinforced constructs

- `range(0, n, chunk)` — start positions of each chunk.
- `list += [...]` — append every item.
- `stack.append(v); stack.pop()` — a list as a stack.
- `np.partition(D, k - 1, axis=1)` — the k smallest per row, unsorted.
- `x if cond else y` — a conditional expression.
- `a, (b, c) = t` — nested unpacking.

End of Day 121 syntax reference. For the experiments and what they showed, see the Day 121 complete guide.